Gestió de Bases de Dades

UD 1 - Introduction to databases

1r CFGS ASIX/DAW
Teacher: Toni de Andrés

Emili Darder

BLOCK 1 – DATABASE ENGINEERING

LESSON 1 – Introduction to databases

LESSON 2 – Logical Design: Entity–Relationship Model

LESSON 3 – Logical to Physical Design: Relational Model

BLOCK 2 – DATABASE MANAGEMENT

LESSON 4 – Physical Design: Data Definition Language

LESSON 5 – Implementation Stage: Data Management Language

BLOCK 3 – DATABASE MAINTENANCE

LESSON 6 – Implementation Stage: SQL Embedded in a Formal Language

LESSON 7 – Maintenance Stage: Triggers

LESSON 8 – Maintenance Stage: Redesigning the DB Model

LESSON 9 – Data Security Management

UD1 - Objectives

After completing this lesson, you should be able to:

- Define a database (or better ... a DBMS)
- Define a Relational database (and RDBMS) and its primary goals
- Identify other data models based on Flat files
- Identify the differences between databases others data models such as spreadsheets
- Recognize the importance of Engineering in IT systems
- Place the analysis and design of data in the engineering process

UD1 - INDEX

UD1 - Introduction to databases - From Data to Information

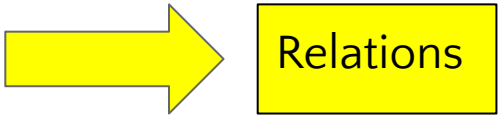
DATA

Single collection of symbols assumed as facts.

EXAMPLE

Student: Xavier Marcus

Subject: DB Management



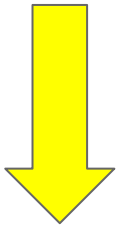
UD1 - Introduction to databases - From Data to Information

INFORMATION

Pieces of data once processed.

EXAMPLE

"Xavier Marcus" is a student of *"DB Management"* subject.



These assertion give us a plus that does not give the assertions seen before.

UD1 - Introduction to databases - From Data to Information

DB

Placeholder where data remains.

DBMS

A software package designed to define, manipulate, retrieve and manage data in a database.

UD1 - Introduction to databases - From Data to Information

DATA MODEL

Describes the way how data is logically structured (and stored) in a DBMS, thereby describes: data , data relationships, data semantics and data constraints.

Types of Data models

- Based on flat files
  - Flat or binary non-unmarked files
  - XML/JSON documents
  - Spreadsheets
- Based on a system which manage a DB
  - Relational or Object relational
  - Distributed
  - NoSQL

UD1 - Introduction to databases - From Data to Information -

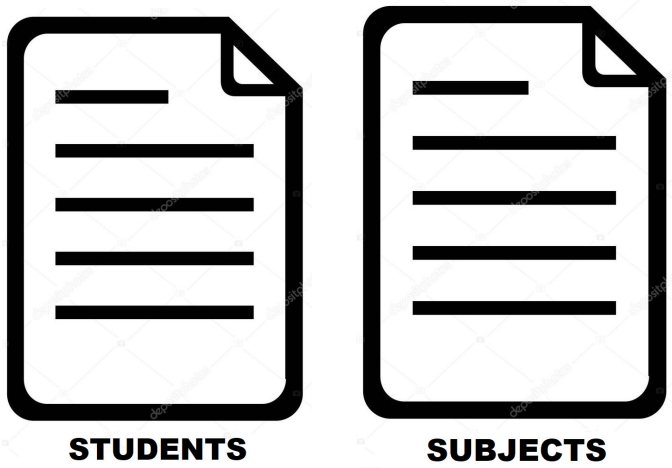
DATA MODEL

- **Flat or binary non-unmarked files**

  Are simply files containing text. No **bold**, *italic*, *different font faces*, or other special font tricks. Just text. Flat files can have a binary organization.

  Access to the data can be one of the following:

  - Sequential chained
  - Sequential indexed
  - Sequentially indexed-chained



UD1 - Introduction to databases - From Data to Information -

DATA MODEL

Flat files work well if ...	Flat files don't work well if ...
• ... values are fairly small and simple • values don't change too often • ... you want to be able to change values with a simple text editor • ... you want to be able to distribute settings by copying files to new locations • ... you want to keep a simple historical list of previous values, such as a list of previous daily memos or welcome messages • ... you want to use tools to quickly compare two files	• ... you need to perform complex searches through the values • ... values change often • ... you don't want others to be able to view and modify the values easily • ... values are hierarchical

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

- **XML, JSON Documents**

  Represents a hierarchical structure of labeled values stored in text documents. They are commonly used as a lightweight data-interchange format:

  - **XML (eXtensible Markup Language)** itself doesn't provide any tools for building, searching, updating, ~~validating~~ (through XSD files), or otherwise manipulating data.
  - **JSON (JavaScript Object Notation)** is built on two structures:
    - A collection of name/value pairs
    - An ordered list of values.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

```
<course id="2018/19">
    <subjects>
        <subject id="1" name="MATHS" />
        <subject id="2" name="ENGLISH" />
        <subject id="3" name="IT />
    </subjects>
    <students
    ...
    </students>
</course>
```

```
<students>
    <student id="1" name="XAVIER" surname="MARCUS">
        <subjects>  <subject id="1" />
                    <subject id="3" />
        </subjects>
    </student>
    <student id="1" name="MIQUEL" surname="GARCIA">
        <subjects>  <subject id="1" />
                    <subject id="2" />
        </subjects>
    </student>
</students>
```

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

```
{
  "course":
        "id": "2018/19",
        "subjects":  {
                "Subject": {
                "id":"1",
                "name": "MATHS" }
              }
              { ... }
              { ... }
        }
        ...
}
```

```
{
  "student":
        "id": "1",
        "name": "XAVIER",
        "surname": "MARCUS",
        "subjects": ["1", "2", "3"]
}
...

...
```

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

XML/JSON work well if ...	XML/JSON documents don't work well if ...
... data is naturally hierarchical There are available lots of XML/JSON tools or libraries providing the features you need (import, export, format transformation ...) ... you want the kinds of validation that schema files can provide. ... you want to import and export the data in products that understand XML/JSON.	... you use non-hierarchical data such as networks (described in the following section) ... you need more complex data validation than schema files can provide ... you need to perform relational rather than hierarchical queries The database is very large so rewriting the entire file to update a small bit of data in the middle is cumbersome ... you need to allow multiple users to frequently update the database without interfering with each other

UD1 - Introduction to databases - From Data to Information -

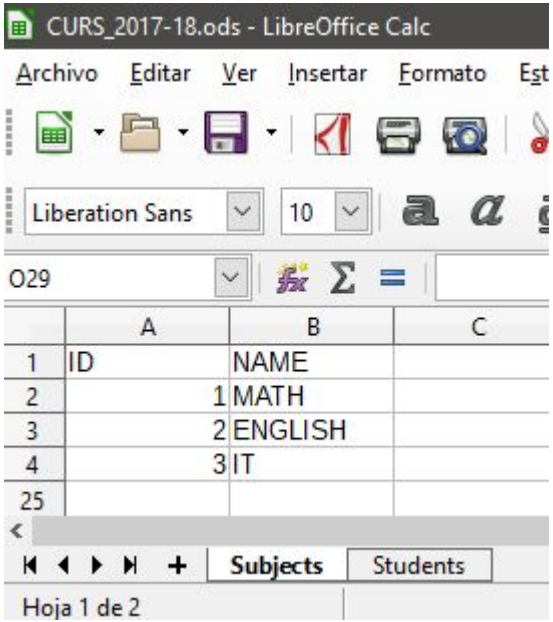
DATA MODEL

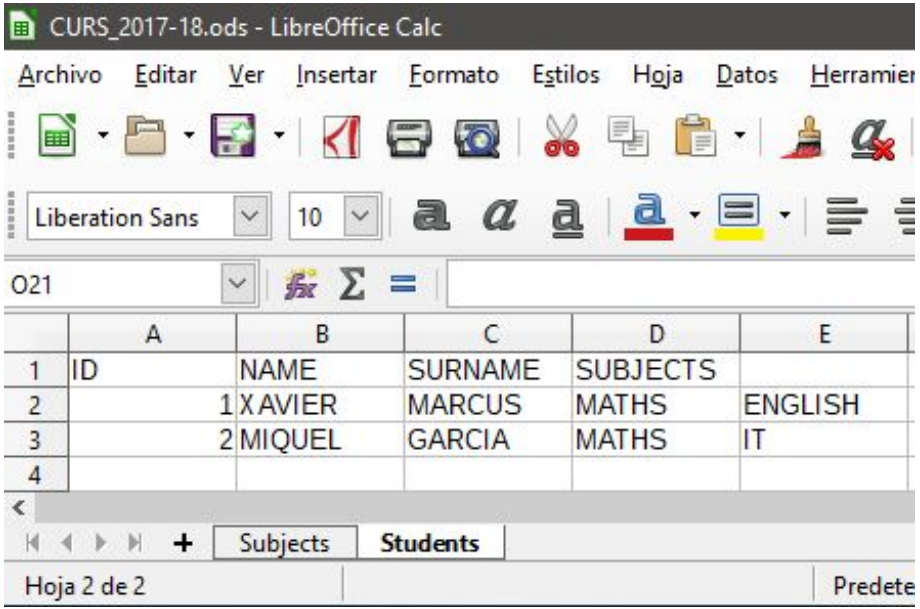
- **Spreadsheets**

  Display rows and columns of data. They allow the user to create formulas that depend on other data, make charts and graphs to visualize the data, print the data, and import and export the data in text and other formats. A spreadsheet may also support relatively sophisticated analysis tools such as statistical functions and iterated solution finding (basically making a bunch of guesses to see which ones work best). Spreadsheets are widely used by end-users due to their facility of use.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL





UD1 - Introduction to databases - From Data to Information -

DATA MODEL

DBMS versus spreadsheets ...

PRIMARY KEY PURPOSE

- Spreadsheets are great for one-time events that do not require tracking of many different aspects. DBMS are for longer projects where data is likely to be used again and again.

NUMBER OF USERS

- The number of access in spreadsheets is reduced (single user or nowadays shared for Google documents) in comparison of DBMS (multi-user access for a single data).

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

DBMS versus spreadsheets ...

Data organization

- Spreadsheet assumes data as flat, DBMS have a complex data structure.

Complexity of data

- Spreadsheet data is simple. It can be easily added to a single table. DBMS house a lot of different types of data that all have some relationship to the other data in the database.

Repetition of data

- Data is assumed to can be repeated in spreadsheets. That does not have a great impact in the way we are used to use these kind of documents. On the contrary, the main purpose of a DBMS is not repeat data in its structure.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

Spreadsheets work well if ...	Spreadsheets don't work well if ...
● ... data fits naturally in a simple tabular format. ● ... you need to visualize the data in charts and graphs. ● ... end users are comfortable with spreadsheets. ● ... end users want to be able to experiment with the data on their own.	● ... complex relationships among the values on different worksheets are needed. ● ... you need to perform complex calculations that a spreadsheet cannot easily handle. ● ... data validation is needed. ● ... you need to perform complex queries. ● ... you need to update large amounts of data automatically.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

- **Relational**

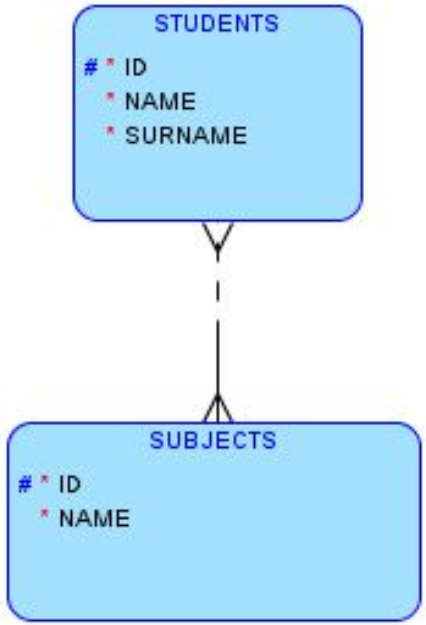
  A Relational database is structured in a set of records set in relations (named Tables holding rows and columns).

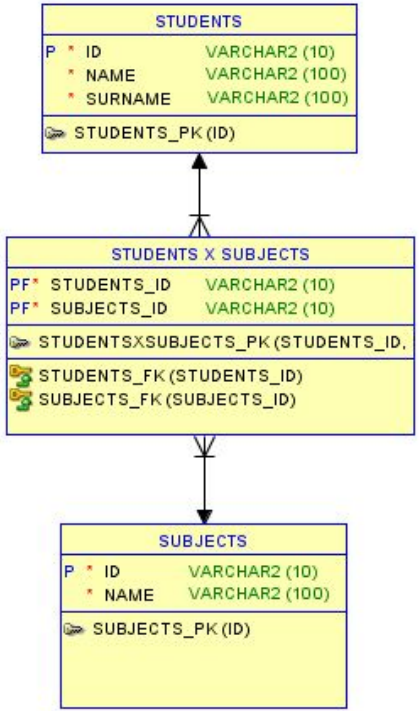
  Each row is related to data on a particular item of an entity. Relational databases use Structured Querying Language (SQL) and Programing Language SQL (PL/SQL) to query and manage data. These languages are simple to use.

  There are a lot of languages that are capables to embed SQL into them: PHP, JAVA, PYTHON and many many others are good examples.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL





UD1 - Introduction to databases - From Data to Information -
DATA MODEL

RDBMS work well if ...	RDBMS don't work well if ...
• ... we want guarantee ACID properties for data, we can create the database schema with referential integrity at its core. • ... we want our application to handle a lot of complicated queries, database transactions and routine analysis of data. Therefore are indicated for that. • Relational means SQL guaranteed access. There are lots of experts in SQL (and PL/SQL) language.	• ... ACID properties would trigger a performance issue.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

- **Object / Object relational**

  An object database provide features for object languages. An object-relational database (ORD) or object-relational database management system (ORDBMS) is a relational database that provides extra features for integrating object types into the data. Like a relational database, it can perform complex queries relatively quickly. Like an object database, it uses some special syntax to simplify the creation of objects.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

- **Distributed**

  In a distributed database (DDBMS), there are a number of databases that may be geographically distributed all over the world. A distributed DBMS manages the distributed database in a manner so that it appears to be a one single database to users.

  In others words, a DDB is a collection of multiple interconnected databases, which are spread physically across various locations that are communicated via a computer network.

  A DDB can accomplish the rules for a RDBMS in terms of Integrity rules.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

DDBMS work well if ...	DDBMS don't work well if ...
• ... we need to expand the system, so that they are easy to scale. • ... are more reliable than centralized databases. When a component fails, the functioning of the system continues, may be at a reduced performance. • ... data is distributed in an efficient manner, then user requests can be met from local data itself, thus providing faster response. • ... the communication costs for data manipulation can be minimized.	• Need for complex and expensive software. • Even simple operations may require a large number of communications and additional calculations to provide uniformity in data across the sites. • Improper data distribution often leads to very slow response to user requests. • ... updating data in multiple sites pose problems of data integrity is needed.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

- **NoSQL or Non-Relational**

  A NoSQL systems store and manage data in ways that allow high operational speed and great flexibility on the part of the developers. In fact, while relational databases have traditionally sacrificed performance and scalability for the ACID (Atomicity, Consistency, Isolation and Durability) properties behind reliable transactions, NoSQL databases have largely neglected those ACID guarantees in favor of speed and scalability.

  Each NoSQL database tends to have its own syntax for querying and managing the data. MongoDB, for instance, sends JSON objects over a binary protocol, by using a command-line interface or a language library.

UD1 - Introduction to databases - From Data to Information -
DATA MODEL

NoSQL databases work well if ...	NoSQL databases don't work well if ...
• ... we want fast access to data, and we are more concerned with speed and simplicity of access than reliable transactions or consistency. • A non-relational database just stores data without explicit and structured mechanisms to link data from different tables (or buckets) to one another. • Data can be stored in a schema-less so changes in data relationship does not affect data model.	• ... we want to maintain consistency in the database (referred to Referential integrity) • Many applications call for the kinds of constraints, consistency, and safeguards that Relational databases provide. In those cases, some "advantages" of NoSQL may turn to disadvantages: there are no joins like there would be in relational databases • Another downside to NoSQL is the relative lack of expertise. The demand for NoSQL expertises is growing, but it's still a fraction of the market for conventional SQL expertises.

UD1 - Introduction to databases - From Data to Information -

DATA MODEL

SCHEMA

Assumed that data is a collection of symbols, schema refers to the type of them: characters, numbers, date, boolean, ... or another more complex data types such as record, objects ...

In a Relational Data Model, Schema is the way how relations are organized and how are related between them.

In other words, schema in a data model could mean the skeleton where data is seated.

UD1 - Introduction to databases - From Data to Information

LANGUAGES OF A DBMS

DDL - Data Definition Language is ...

A set of statements which allows to set up a schema for a DBMS.

DML - Data Manipulation Language is ...

A set of statements which give us the way to query and manage data of a DBMS.

Turning *data* into *information*

Inserting, Updating and Deleting data

DCL - Data Control Language is ...

A set of statements which give us the way to access users of a DBMS to data.

UD1 - Introduction to databases - From Data to Information

At this moment you have to fix the following ideas:

1. This year we will learn how transform data into information through DBMS. Also we will work HARD with Relational Database Model to organize data and the information that can be inferred from that by using DBMS. Therefore, we will manage SQL and PL/SQL for the CRUD operations.

1. Furthermore, you will learn (in another subject) how a XML/JSON documents (which are a kind of structured labeled flat files) work. Meanwhile, this subject is related to RDBMS.

1. Henceforth you must known how many kinds of DB Models consider depending on the way we want to work within data. It is good to know every type of Data model as well as their strengths and weaknesses.

UD1 - INDEX

UD1 - Introduction to databases - Goals of a DBMS - Capabilities of a DBMS

A DBMS is demanded for providing next capabilities ...

- Massive,
- Persistent,
- Safe,
- Multi-user,
- Convenient,
- Efficient,
- Reliable.

Flat files and spreadsheets do not cover this capabilities at all.

UD1 - Introduction to databases - Goals of a DBMS - Capabilities of a DBMS

A DBMS is demanded for providing next capabilities ...

- **MASSIVE**:
  - Handle a large amount of data. It exceeds the capability of process of a human process much than we can imagine.
- **PERSISTENT**:
  - Data overlives the execution of a program. Multiple programs can access to the same data at the same time.
- **SAFE**:
  - In the sense that software programs and DBMS are separated systems, They are safe not to suffer software failures because they implement securized systems by their own such a: malicious user access,

UD1 - Introduction to databases - Goals of a DBMS - A DBMS provides next capabilities ...

- **MULTI-USER**:
  - Many applications can access to the same data (or many users of a single application). That's what we call concurrency access. In this case, DBMS have to provide mechanisms to stay data consistent mantening exclusive access to data. In this case, performance of DBMS would not slow down.
- **CONVENIENT**:
  - In the sense that they are easy to work with huge amount of data by the fact that DBMS maintain physical and logical access to data is separated and the way that this data is stored and how it can change by redesigning by IT Engineering.
  - Easy to query data using a simple (non-algorithm) high-level language system (SQL and PL/SQL).

UD1 - Introduction to databases - Goals of a DBMS - A DBMS provides next capabilities ...

- **EFFICIENT**:
  - DBMS show an efficient behaviour by their own, but Administrator must work on that direction. So when we talk in DBMS terms one of the most important thing is :
    i. First performance,
    ii. Second performance and
    iii. Again performance.
- **RELIABLE**:
  - ... in the sense that it is critically important for many others IT systems that are up all the time for the things that DBMS provides. It is called a High availability system: 24x7x365 service, 24 hours, 7 days a week, 365 days a year.

UD1 - Introduction to databases - Goals of a DBMS - ACID properties in DBMS ...

A transaction in a DBMS is ...

A transaction is a single logical unit of work which accesses and possibly modifies the contents of a database. Transactions access data using read and write operations. In other words, a transaction is a group of operations that acts as a single unit.

A transaction acts in isolation from other transactions make them durably stored.

UD1 - Introduction to databases - Goals of a DBMS - ACID

properties in DBMS ...

ACID is an acronym of ...

- Atomicity,
- Consistency,
- Isolation,
- Durability.

Let's go for every each one ...

*https://www.geeksforgeeks.org/acid-properties-in-dbms/

UD1 - Introduction to databases - Goals of a DBMS -

ACID properties in DBMS ...

ATOMICITY

The entire transaction (that means: every operation which is involved on) takes place at once or it doesn't happen at all. Each transaction is considered as one unit and either runs to completion or is not executed at all.It involves next operations:

- **Rollback**: if a transaction rollbacks, changes made to database are not visible.
- **Commit**: If a transaction commits, changes made are visible

Before: X : 500	Y: 200
Transaction T	
T1	**T2**
Read (X) X: = X – 100 Write (X)	Read (Y) Y: = Y + 100 Write (Y)
After: X : 400	Y : 300

UD1 - Introduction to databases - Goals of a DBMS -

ACID properties in DBMS ...

CONSISTENCY (or Referential Integrity)

The links associated with data must be maintained once a transaction has been executed.

T is a Number after and before this transaction

Total **before T** occurs = **500 + 200 = 700**.

Total **after T occurs** = **400 + 300 = 700**.

In RDBMS we may talk about **Referential Integrity which enforces** next rules:

1. **No Unattended Child rule** or **a No Orphans rule**,
2. If any deletion occurs, a **Cascade deletion** must occur,
3. If any update occurs, a **Cascade updation** must occur.

UD1 - Introduction to databases - Goals of a DBMS -

ACID properties in DBMS ...

ISOLATION

This property ensures that multiple transactions can occur concurrently without leading to inconsistency of database state. In this case, transactions occur independently without interference. In others words, changes occurring in a particular transaction will not be visible to any other transaction until that particular change in that transaction has been COMMITED.

T	T''
Read (X) X: = X*100 Write (X) Read (Y) Y: = Y – 50 Write	Read (X) Read (Y) Z: = X + Y Write (Z)

UD1 - Introduction to databases - Goals of a DBMS -

ACID properties in DBMS ...

DURABILITY

This property ensures that once the transaction has completed execution, the updates and modifications to the database are stored in and written to disk and they persist even is system failure occurs. In other words, the effects of the transaction, thus, are never lost.

T	T''
Read (X) X: = X*100 Write (X) Read (Y) Y: = Y – 50 Write	Read (X) Read (Y) Z: = X + Y Write (Z)

Consequently, onces Commit occurs, Rollback has no longer effect. Equally valid on the contrary.

UD1 – Introduction to databases – Goals of a DBMS – DBMS are good at ...

To Sum up, DBMS are extremely prevalent today ...

- They are behind websites, banking systems, telecommunications, deployments a lot of IT systems, scientific experiments and much, much more ...

- Nowadays it is quite difficult to imagine an IT system without a DBMS behind. Or at least a data management behind.

UD1 - Introduction to databases - Goals of a DBMS - DBMS are good at ...

DBMS contains information about a particular enterprise ...

- Collection of interrelated data
- Set of programs to access the data
- An environment that is both convenient and efficient to use

Database Application examples

- Banking: all transactions
- Airlines: reservations, schedules
- Universities: registration, grades
- Human resources: employee records, salaries, tax deductions
- Sales: customers, products, purchases
- Manufacturing: production, inventory, orders, supply chain
- Online retailers: order tracking, customized recommendations

In essence Databases touch all aspects of our lives (more than it is believed) !!

UD1 - Introduction to databases - Goals of a DBMS

At this moment you have to fix the following ideas:

1. DBMS are separated systems from others which are living in a huge IT system.
2. DBMS provide some capabilities we have summarized. It is great to have these features provided if you have an application you want to build.
3. ACID properties are present on RDBMS.

1. Seriously ?, Do you want to work hard with data ? ... You need a DBMS instead of spreadsheets or any kind of flat files ...

UD1 - INDEX

UD1 - Introduction to databases - Architecture of a DBMS - How does it work ?

Based on Client-Server Model

1

```
SELECT id, name, surname
FROM students
```

ID	NAME	SURNAME
36	Bob	Stevens
37	Alice	Campbell
...	...	...

2

Oracle tcp/1521
MySQL tcp/3306
PostgreSQL tcp/5432

UD1 - Introduction to databases - Architecture of a DBMS - Data description level

DBMS is said to must accomplish the three-level architecture (known as **ANSI/SPARK Architecture**) in order to separate the user's view.

The three levels are:

1. **External level**:

   Represents the different ways in which users might understand (view) the same data

1. **Conceptual level**:

   A single data structure that is internally consistent and which supports the different external views

1. **Internal level**:

   A description of the physical arrangement of data in files and devices

UD1 - Introduction to databases - Architecture of a DBMS - Data description level

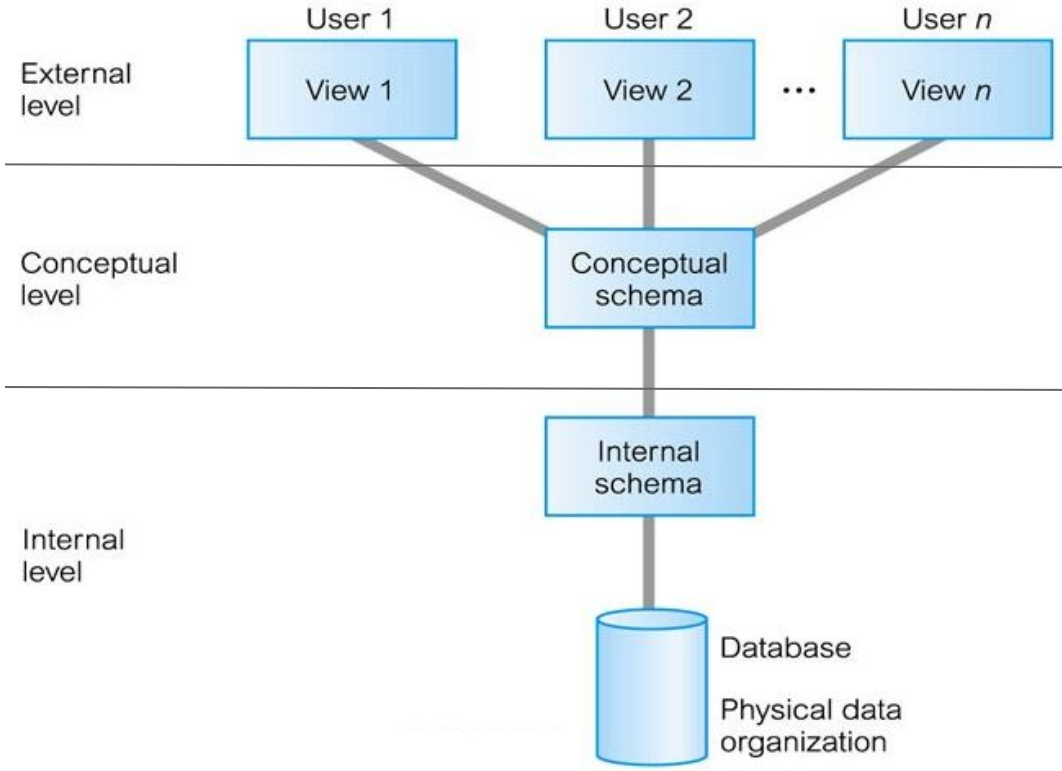
When users interact with the DB, they do it through the relationship between the data levels which are established internally by how a DBMS works. That is what we call **mapping** of data levels.

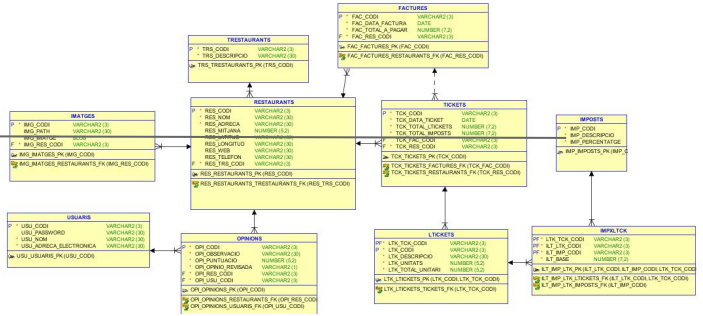
There are the two types of mappings:

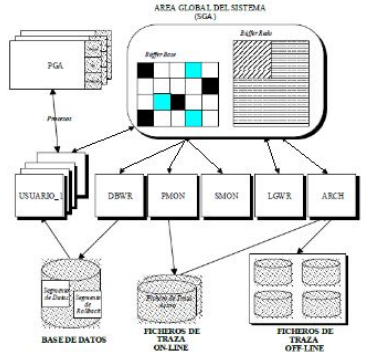
1. External ⇆ Conceptual Mapping
2. Conceptual ⇆ Internal Mapping

Everything together is unsighted for users.

UD1 - Introduction to databases - Architecture of a DBMS - Data description level







UD1 - Introduction to databases - Architecture of a DBMS

At this moment you have to fix the following ideas:

1. DBMS work under a model based on Client-Server Architecture.

1. DBMS keep a three-level structure that we have to retain: external, conceptual and internal levels. When a user requires information from the DB, the MS provides this information through a mapping data.

1. DBMS works as a complex system we will learn next year.

UD1 - INDEX

UD1 - Introduction to databases - Users of a DBMS

- **DBMS ADMINISTRATOR**
  - Who loads data , keeps DB running smoothly by tuning DBMS appropriately
- **DBMS IMPLEMENTER**
  - Who builds the physical system
- **DBMS DESIGNER**
  - Who establish the schema for a DB
- **APPLICATION DEVELOPER**
  - Who makes programs that operate on the DB

UD1 - Introduction to databases - Users of a DBMS

The DBMS ADMINISTRATOR coordinates all the activities of the database system; the database administrator has a good understanding of the enterprise's information resources and needs. Database administrator's duties include:

- Schema definition,
- Storage structure and access method definition,
- Schema and physical organization modification,
- Granting user authority to access the database,
- Specifying integrity constraints,
- Acting as liaison with users,
- Monitoring performance and responding to changes in requirements ensuring a high performance.

UD1 - Introduction to databases - Users of a DBMS

At this moment you have to fix the following ideas:

1. Different users provide different type of jobs everyone of them related to IT.

UD1 - INDEX

UD1 - Introduction to databases - Most popular DBMS

Pros and Cons

UD1 - INDEX

UD1 - Introduction to Modeling - The life cycle of Applications Development

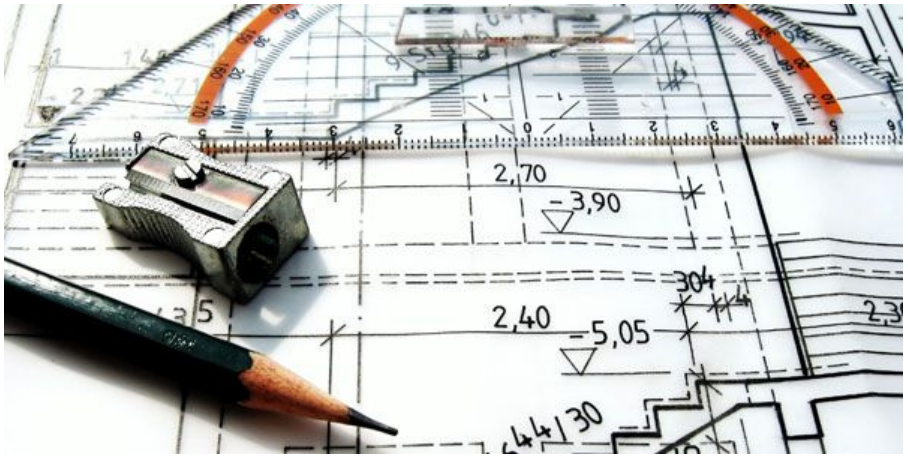
Software engineering is devoted to the preliminary, systematic and disciplined study of the development of computer systems.

A computer system is a structure capable of storing and processing information:

- Formed by software, hardware and human resources
- Needs to be integrated into an organization

UD1 - Introduction to Modeling - The life cycle of Applications Development

Software engineering has several well-known and standardized methods to capture an idea before implementing it.



UD1 - Introduction to Modeling - The life cycle of Applications Development

The lack of study of a computer system may in future lead to problems that can hardly be resolved or the solution will be either more expensive than the initial solution.

UD1 - Introduction to Modeling - The life cycle of Applications Development

In Software Engineering, the development methodology defines the framework used to structure, plan, control the development process of the information system to be developed.

There are different methodologies to develop them:

- CADM (CASE Application Development Method)
- Metric
- Merisse
- SSADM (Structured Method of Analysis and Systems Design)
- Agile methodologies
  - Scrum

UD1 - Introduction to Modeling - The life cycle of Applications Development

Anàlisi	**ENTENDRE el problema**
Disseny	**PENSAR les possibles solucions del problema**
Construcció	**IMPLEMENTAR** una solució en l'**Entorn de desenvolupament** i l'**Entorn de producció**

UD1 - Introduction to Modeling - The life cycle of Applications Development

The process of designing the general structure of the database involves:

- A **Logical Design** – Deciding on the database schema. Database design requires that we find a "good" collection of relation schemas.
  - Business decision – What attributes should we record in the database?
  - Computer Science decision – What relation schemas should we have and how should the attributes be distributed among the various relation schemas?
- **A Physical Design** – Deciding on the physical layout of the database

Recursos de la UD

Additional resources

- Video (SU): Introduction to databases
- Video: Data vs Information vs Acknowledge

Assignment

- ACT1.1 – Introduction to databases
- ACT1.2 – Most popular DBMS

Bibliografia

- **Stanford University**; Stanford Databases MOOC; Introduction and relational Databases - MOOC DB1
- **Universitat de les Illes Balears**; Bartomeu Vives; Funcionamiento de Oracle
- **Jorgesanchez.net**; Jorge Sanchez; Gestión de BD
- **Geeks for geeks**; ACID properties for a DBMS
- **Pluralsight**; Jacqueline Homan; Relational vs. non-relational databases: Which one is right for you?
- **Tutorialspoint**; Distributed DBMS - Distributed Databases
- **Inforworld**; Serdar Yegulalp; What is NoSQL? NoSQL databases explained
- **Keycdn**; Cody Arsenault; The Pros and Cons of 8 Popular Databases

Sou lliure de:

Compartir — copiar i redistribuir el material en qualsevol mitjà i format
Adaptar — remesclar, transformar i crear a partir del material

El llicenciador no pot revocar aquestes llibertats, sempre que seguiu els termes de la llicència.

Amb els termes següents:

Reconeixement — Heu de reconèixer l'**autoria de manera apropiada**, proporcionar un enllaç a la llicència i **indicar si heu fet algun canvi**. Podeu fer-ho de qualsevol manera raonable, però no d'una manera que suggereixi que el llicenciador us dóna suport o patrocina l'ús que en feu.
NoComercial — No podeu utilitzar el material per a **finalitats comercials**.
CompartirIgual — Si remescleu, transformeu o creeu a partir del material, heu de difondre les vostres creacions amb la **mateixa llicència** que l'obra original.

No hi ha cap restricció addicional — No podeu aplicar termes legals ni **mesures tecnològiques** que restringeixin legalment a altres de fer qualsevol cosa que la llicència permet

https://creativecommons.org/licenses/by-nc-sa/4.0/legalcode